Formulários para a web

Caderno do estudante

Professor Fabrício Rodrigues

Curso Técnico em Informática

16 de outubro a 18 de dezembro de 2026

Explicações, exemplos, atividades e mapas. Os trechos de código são pistas de estudo, não substituem o arquivo completo da prática.

9 encontros de 100 minutos. 18 aulas de 50 minutos. 15 horas de relógio. Sem aula em 20/11.

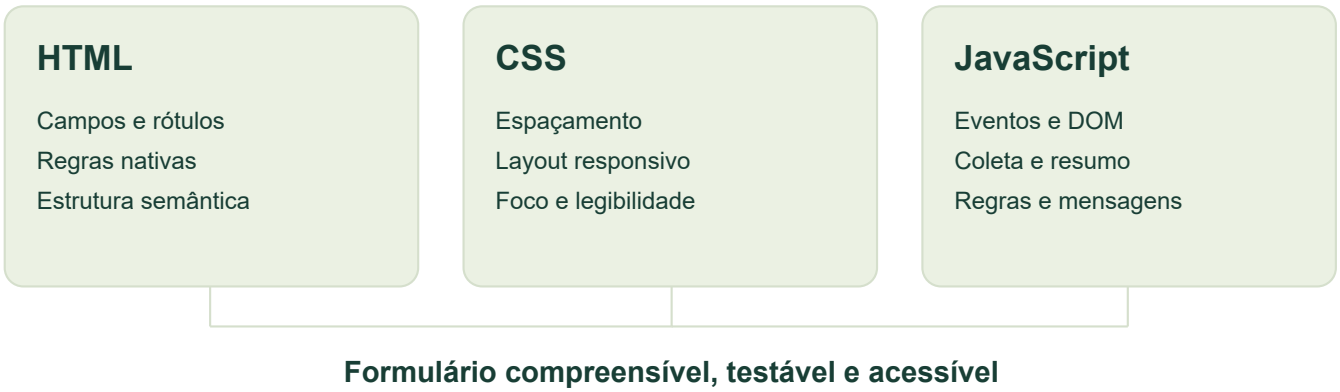
Como usar este caderno

Antes da aula, leia o objetivo e a retomada. Durante a prática, altere um comportamento por vez e registre o teste. Depois, use o checklist para explicar o que conseguiu construir.

O projeto do período

Vamos desenvolver um chamado de suporte ao laboratório com dados fictícios. A versão final coleta informações, verifica regras e apresenta um resumo local. O projeto informa que o envio é simulado.

Mapa dos conhecimentos



Arquivos para estudo sem internet

Extraia o pacote do aluno e abra index.html. Para a primeira prática, use pratica/diagnostico-base/index.html. Para o projeto, use pratica/formulario-base/index.html. O arquivo pratica/formulario-modelo.html permite testar os comportamentos após a tentativa própria.

O PDF pode ser lido ou impresso. Os links das referências exigem internet quando forem abertos. Os códigos e exemplos do pacote funcionam sem conexão e sem instalação.

Mapa do período

Todas as datas abaixo são sextas-feiras de 2026. Cada encontro tem duas aulas de 50 minutos.

Data	Tema	Evidência da aprendizagem
16/10	O que já sabemos sobre a web?	Ficha individual, página corrigida e primeiro formulário com dois campos.
23/10	Campos com significado	Versão 1 do formulário com campos e restrições HTML.
30/10	Um formulário legível em qualquer tela	Versão 2 com layout responsivo e teste de teclado.
06/11	Os dados aparecem na página	Versão 3 que apresenta um resumo sem recarregar a página.
13/11	Mensagens que ajudam a corrigir	Versão 4 com uma regra de confirmação de contato e mensagens claras.
27/11	O caminho dos dados	Especificação curta do projeto e mapa do percurso dos dados.
04/12	Um formulário que reage ao contexto	Versão integradora com campo complementar condicional.
11/12	O projeto nas mãos de outra pessoa	Projeto revisado, publicado e acompanhado de registro de testes.
18/12	O que conseguimos construir e explicar	Projeto final, explicação individual e comparação com o diagnóstico inicial.

20/11: sem aula. A retomada acontece em 27/11.

ENCONTRO 01 / 16/10 / Diagnóstico

O que já sabemos sobre a web?

Diagnóstico de HTML, CSS e JavaScript e primeiro contato com formulários

O que você deverá conseguir

• Ler trechos de HTML, CSS e JavaScript e explicar o comportamento esperado.

• Localizar um erro de seletor e registrar como o corrigiu.

• Associar um rótulo a um campo no primeiro formulário.

Ponto de partida

O portal anterior apresenta semântica HTML, Flexbox/Grid, responsividade, DOM, eventos e login simulado. A presença no material indica conteúdos trabalhados, e o diagnóstico verificará o domínio real da turma.

Diagnóstico como ponto de partida

A atividade inicial serve para orientar as próximas aulas. Registre o que consegue explicar sozinho, o que faz com uma pista e o que ainda precisa estudar. Uma resposta incompleta com raciocínio ajuda o professor a planejar a retomada.

Responda às nove questões antes da conversa coletiva. Na prática, faça pequenas mudanças em uma página conhecida. Depois da tentativa individual, a turma discutirá as estratégias e as dúvidas.

Uma página conhecida, um problema por vez

Na base de diagnóstico, a classe do destaque e os seletores do JavaScript não correspondem ao HTML. Compare os nomes literalmente, leia o console e teste a página após cada alteração. Acrescente um cartão e explique por que escolheu as tags usadas.

A ponte para formulários

Um chamado de suporte precisa receber informações que uma página de divulgação apenas exibe. Começaremos com equipamento e descrição do problema. Os dados do exercício são fictícios e ficam no navegador.

O label descreve o campo. Seu atributo for deve apontar para o id do controle. O name será a chave usada na coleta dos valores.

```
<form id="chamado">
  <label for="equipamento">Equipamento</label>
  <input id="equipamento" name="equipamento">
  <label for="descricao">Descrição do problema</label>
  <textarea id="descricao" name="descricao"></textarea>
  <button type="button">Protótipo em construção</button>
</form>
```

Atividade da aula

1. Responder à ficha individual do diagnóstico.
2. Corrigir a classe CSS e o seletor JavaScript da página-base, incluir um cartão e testar dois cliques.
3. Criar um protótipo com equipamento e descrição, com rótulos associados.
4. Registrar uma facilidade e uma dúvida para a próxima aula.

Checklist de conferência

[ ] Expliquei uma decisão de HTML, uma de CSS e uma de JavaScript.

[ ] Registrei o erro que encontrei e como testei a correção.

[ ] Associei cada label ao id do campo.

Uma descoberta útil

Clicar em um rótulo associado pode levar o foco ao campo. Essa associação também ajuda tecnologias assistivas.

Para ir além

Acrescentar um campo de contato fictício e justificar o tipo escolhido.

Referências da aula

Base anterior: portal do Prof. Ábner Lucas

MDN: seu primeiro formulário

W3C WAI: rótulos dos controles

ENCONTRO 02 / 23/10 / HTML

Campos com significado

Estrutura, rótulos e regras nativas do formulário

O que você deverá conseguir

• Distinguir id, name, type e value.

• Escolher controles adequados a texto, escolha única e múltiplas opções.

• Usar required e restrições coerentes com o dado solicitado.

Ponto de partida

Retomar as dificuldades de HTML observadas em 16/10. A apresentação de um formulário organiza dados e precisa de campos identificáveis.

Um campo tem mais de um nome

id identifica um elemento na página e conecta o label. name identifica o dado coletado. type define o comportamento do controle, e value representa seu valor. Os ids devem ser únicos no documento.

Use rótulos visíveis. Um placeholder pode dar um exemplo curto, mas desaparece durante o preenchimento e não substitui o label.

```
<label for="contato">E-mail de contato fictício</label>
<input id="contato" name="contato" type="email" required>
```

Controles adequados ao problema

Use input text para um código de equipamento como PC-07. Esse código não é uma quantidade para calcular. Use select para escolher uma categoria e textarea para descrever o defeito.

Radio permite uma escolha em um grupo com o mesmo name. Checkbox permite marcar opções independentes. Agrupe opções relacionadas em fieldset, com legend que explique o conjunto.

```
<fieldset>
  <legend>Prioridade do chamado</legend>
  <label><input type="radio" name="prioridade" value="normal" required>
    Normal</label>
  <label><input type="radio" name="prioridade" value="alta"> Alta</label>
</fieldset>
```

Regras que o navegador já conhece

required pede um valor. type=email confere um formato de e-mail, sem provar que a caixa existe. minlength e maxlength delimitam comprimentos. min e max se aplicam, por exemplo, a quantidades numéricas.

Explique a regra antes do erro. Se a descrição precisa de pelo menos 20 caracteres, apresente essa orientação junto ao campo. Comece com validação nativa e teste o formulário sem JavaScript.

```
<label for="descricao">Descrição do problema</label>
<p id="ajuda">Escreva pelo menos 20 caracteres.</p>
<textarea id="descricao" name="descricao" required
  minlength="20" maxlength="500" aria-describedby="ajuda"></textarea>
```

Atividade da aula

1. Definir quais dados são necessários a um chamado fictício.
2. Criar campos de solicitante fictício, contato, equipamento, categoria, prioridade e descrição.
3. Adicionar duas opções independentes em checkbox para sintomas observados.
4. Testar campos vazios, e-mail sem formato adequado e descrição curta.

Checklist de conferência

[ ] Todos os campos têm rótulo e name.

[ ] Os ids são únicos e correspondem aos labels.

[ ] Radio e checkbox representam escolhas adequadas.

[ ] As restrições têm instruções visíveis.

Uma descoberta útil

Um button dentro de um formulário pode atuar como envio quando o tipo não é especificado. Declarar type evita ações acidentais.

Para ir além

Criar uma pequena tabela justificando tipo, name e regra de cada campo.

Referências da aula

MDN: tipos de input

W3C WAI: rótulos dos controles

W3C WAI: agrupamento de controles

MDN: validação de formulários

MDN: elemento button

ENCONTRO 03 / 30/10 / CSS e acessibilidade

Um formulário legível em qualquer tela

Layout, foco, instruções e navegação pelo teclado

O que você deverá conseguir

• Organizar os campos sem prejudicar a ordem de leitura.

• Adaptar o layout ao celular e ao computador.

• Manter foco visível e instruções compreensíveis.

Ponto de partida

Reaplicar o box model e o CSS Grid apresentados no trabalho anterior. Os campos continuam sendo controles HTML nativos.

A ordem começa no HTML

Organize os campos em uma sequência que faça sentido antes de criar colunas. A leitura no celular e a ordem da tecla Tab devem acompanhar essa sequência. O CSS pode mudar a aparência, mas não deve tornar a navegação confusa.

Use largura fluida, espaçamento consistente e fonte legível. Evite definir altura fixa para textarea quando isso impedir a leitura.

```
* { box-sizing: border-box; }
input, select, textarea, button { font: inherit; }
input:not([type="radio"]):not([type="checkbox"]),
select, textarea { width: 100%; padding: .7rem; }
.campo { display: grid; gap: .35rem; }
```

Colunas quando houver espaço

Comece com uma coluna. Adicione duas colunas em telas maiores e mantenha a descrição ocupando a largura necessária. Teste com a janela estreita e com ampliação de texto, em vez de confiar apenas em uma captura de tela.

```
.campos { display: grid; gap: 1rem; }
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .campo-largo { grid-column: 1 / -1; }
}
```

Pistas que não dependem só da cor

A borda vermelha, sozinha, não explica um erro. Escreva o que precisa mudar e mantenha a orientação próxima do campo. Não apague o contorno de foco sem oferecer uma alternativa visível.

Navegue com Tab e Shift+Tab, acione os controles pelo teclado e clique nos labels. Confira se textos continuam legíveis e se a página evita rolagem horizontal.

```
:focus-visible {
  outline: 3px solid #54753a;
  outline-offset: 3px;
}
.ajuda { color: #52634d; font-size: .95rem; }
```

Atividade da aula

1. Aplicar uma identidade visual própria ao formulário existente.

2. Usar uma coluna no celular e duas onde houver espaço.

3. Manter a descrição em uma área ampla e os labels próximos dos campos.

4. Trocar o computador com um colega e testar apenas pelo teclado.

Checklist de conferência

[ ] O formulário funciona em 360px e em uma tela larga.

[ ] A tecla Tab segue uma ordem compreensível.

[ ] O foco é visível em links, botões e campos.

[ ] As instruções não dependem apenas da cor.

Uma descoberta útil

A ordem visual pode mudar com CSS sem mudar a ordem de foco. Isso pode confundir quem navega usando o teclado.

Para ir além

Testar o navegador com zoom de 200% e documentar uma melhoria.

Referências da aula

MDN: estilização de formulários

MDN: fundamentos de CSS Grid

W3C WAI: rótulos dos controles

W3C WAI: notificações e erros

ENCONTRO 04 / 06/11 / JavaScript

Os dados aparecem na página

Evento submit, FormData e resumo da solicitação

O que você deverá conseguir

• Tratar o evento submit no formulário.

• Ler os valores por meio de FormData e names coerentes.

• Atualizar texto no DOM sem interpretar a entrada como HTML.

Ponto de partida

Os eventos de clique e a manipulação do DOM já aparecem no acervo anterior. Agora o evento relevante pertence ao formulário.

O evento pertence ao formulário

Escute submit no form para contemplar o envio pelo botão e pelo teclado. event.preventDefault() cancela a ação padrão desse envio. Ele não salva os dados e não cria um servidor.

Na simulação, o JavaScript mostra um resumo local. Uma mensagem deve dizer que o chamado não foi enviado a um serviço real.

```
const form = document.querySelector("#chamado");
const resumo = document.querySelector("#resumo");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  resumo.textContent = `Equipamento: ${dados.get("equipamento")}`;
});
```

As chaves vêm do name

new FormData(form) coleta os controles elegíveis. Um campo sem name não aparece como uma entrada com a chave esperada. Checkbox desmarcado e controle disabled não entram normalmente nessa coleta.

Use get para um valor e getAll quando vários controles compartilham o mesmo name, como uma lista de sintomas. Uma string pode receber trim para retirar espaços no começo e no fim.

```
const dados = new FormData(form);
const descricao = String(dados.get("descricao") || "").trim();
const sintomas = dados.getAll("sintomas");
console.log(descricao, sintomas);
```

Um resumo que continua sendo texto

Use textContent para exibir os valores digitados. Assim, uma entrada parecida com uma tag aparece como texto. Evite construir HTML diretamente com o conteúdo preenchido pelo visitante.

Uma região de status pode avisar que o resumo mudou. O teste precisa verificar o conteúdo exibido, o caminho pelo teclado e a ausência de erros no console.

```
<p id="resumo" role="status" aria-live="polite"></p>
```

Atividade da aula

1. Escutar submit com addEventListener no form.
2. Coletar equipamento, categoria, prioridade e descrição.
3. Mostrar um resumo com textContent e aviso de simulação.
4. Testar envio pelo botão e pela tecla Enter em um campo apropriado.

Checklist de conferência

[ ] O envio não recarrega a página na simulação.

[ ] FormData encontra os names definidos no HTML.

[ ] Os checkbox usam getAll quando necessário.

[ ] O resumo apresenta os dados como texto e informa a simulação.

Uma descoberta útil

Um campo pode ter id e aparecer na tela, mas não ser coletado pela chave esperada quando falta name.

Para ir além

Acrescentar um contador de caracteres com evento input, mantendo a regra minlength no HTML.

Referências da aula

MDN: evento submit

MDN: uso de FormData

MDN: atributo disabled

MDN: textContent

ENCONTRO 05 / 13/11 / Validação

Mensagens que ajudam a corrigir

Regras complementares, erros e casos de teste

O que você deverá conseguir

• Distinguir restrição HTML e regra complementar em JavaScript.

• Limpar uma mensagem personalizada quando o valor muda.

• Testar entradas válidas, inválidas e limites.

Ponto de partida

Retomar required, minlength e type=email. A regra nova compara dois campos, sem substituir as regras que o HTML já oferece.

Uma regra que compara dois campos

Acrescente um campo para repetir o contato fictício. A confirmação deve coincidir com o primeiro contato. Deixe a regra explícita e apresente a ajuda antes da tentativa de envio.

O exemplo trabalha uma comparação simples de strings. A atividade usa dados fictícios, sem senhas ou documentos pessoais. A validação do navegador ajuda o preenchimento, mas um sistema real também verifica os dados no servidor.

Mensagem personalizada precisa ser limpa

setCustomValidity recebe uma mensagem quando o valor não atende à regra. Uma string vazia limpa o erro personalizado. Atualize a regra quando qualquer um dos dois contatos mudar para não manter um erro antigo.

A validação nativa pode impedir o evento submit quando há um campo inválido. Por isso, preparar e atualizar a regra no evento input evita depender apenas do submit.

```
const contato = document.querySelector("#contato");
const confirmar = document.querySelector("#confirmar");
function conferirContato() {
  const diferente = confirmar.value.trim() !== contato.value.trim();
  confirmar.setCustomValidity(diferente ? "Repita o mesmo contato." : "");
}
contato.addEventListener("input", conferirContato);
confirmar.addEventListener("input", conferirContato);
```

Teste a regra e a recuperação

Verifique campo vazio, formato inválido, valores diferentes e correção depois de uma falha. O formulário deve permitir nova tentativa sem exigir que a pessoa reescreva os campos corretos.

Registre entrada, resultado esperado, resultado observado e ajuste realizado. Uma descrição de erro deve explicar o próximo passo, e não apenas dizer “inválido”.

```
if (!form.reportValidity()) {
  return;
}
// Depois de validar, gerar o resumo local.
```

Atividade da aula

1. Criar a confirmação do contato fictício com label e name.
2. Comparar os campos e limpar o erro personalizado após a correção.
3. Exibir uma orientação legível junto ao campo.
4. Registrar pelo menos quatro testes, incluindo a recuperação do erro.

Checklist de conferência

[ ] As restrições HTML continuam funcionando.

[ ] A regra de comparação atualiza quando os valores mudam.

[ ] O erro desaparece depois da correção.

[ ] Meus testes incluem ao menos um caso inválido e sua recuperação.

Uma descoberta útil

Uma mensagem personalizada não vazia mantém o controle inválido. Alterar o texto do campo, sozinho, não apaga a mensagem definida pelo código.

Para ir além

Relacionar uma mensagem própria ao campo por aria-describedby e testar sua clareza com um colega.

Referências da aula

MDN: validação de formulários

MDN: setCustomValidity

W3C WAI: notificações e erros

MDN: evento submit

ENCONTRO 06 / 27/11 / Web e projeto

O caminho dos dados

GET, POST, HTTPS e definição do projeto integrador

O que você deverá conseguir

• Explicar cliente, requisição e servidor em linguagem simples.

• Distinguir parâmetros de GET e corpo de POST.

• Definir o que o protótipo local faz e o que exigiria backend.

Ponto de partida

Não haverá aula em 20/11. Reabrir a versão anterior e conferir o resumo antes de apresentar conceitos novos.

Uma requisição precisa de destino

O navegador pode enviar um formulário a uma URL indicada por action. method define como essa requisição se organiza. Em GET, os campos enviados aparecem normalmente na consulta da URL. Em POST, os dados ficam no corpo da requisição.

POST não significa criptografia. HTTPS protege o tráfego entre os pontos da conexão. Um serviço real precisa receber, validar e tratar os dados.

```
<!-- Exemplo conceitual: esta rota não existe no WebLab. -->
<form action="/api/chamados" method="post">
  <!-- controles com name -->
</form>
```

O limite do protótipo está explícito

GitHub Pages hospeda conteúdo estático. O nosso projeto usa HTML, CSS e JavaScript no navegador e simula o envio. Publicar esses arquivos não cria automaticamente uma API, um banco de dados ou uma caixa de chamados.

Um backend futuro receberia a requisição, faria novas validações, aplicaria regras e retornaria uma resposta. Esse desenvolvimento fica fora das 15 horas deste percurso.

Requisitos do projeto integrador

O produto final será um formulário de suporte para um laboratório fictício. Ele deve organizar campos, orientar o preenchimento, validar entradas e mostrar um resumo local acessível.

Defina uma categoria de problema que abra um campo complementar, como “outro”. Escreva seus casos de teste e as partes que cada aluno consegue explicar, mesmo quando houver colaboração.

Atividade da aula

1. Desenhar o percurso do preenchimento ao resumo local.
2. Marcar no mapa onde entraria um servidor real.
3. Escrever os requisitos do projeto integrador e os campos necessários.
4. Escolher seis casos de teste e planejar as melhorias restantes.

Checklist de conferência

[ ] Expliquei uma diferença entre GET e POST.

[ ] Reconheci que POST sozinho não cifra dados.

[ ] Meu projeto declara envio simulado.

[ ] Os requisitos descrevem comportamentos que consigo testar.

Uma descoberta útil

Um endereço com parâmetros de busca pode ser compartilhado como link. Esse é um uso comum de GET em pesquisas.

Para ir além

Inspecionar uma requisição de busca pública nas ferramentas do navegador, sem enviar dados pessoais.

Referências da aula

MDN: envio de dados

GitHub: funcionamento do Pages

MDN: seu primeiro formulário

ENCONTRO 07 / 04/12 / Integração

Um formulário que reage ao contexto

Campos condicionais e organização do código

O que você deverá conseguir

• Mostrar um campo quando a escolha exige mais detalhes.

• Manter hidden, disabled e required coerentes.

• Organizar a lógica em funções pequenas e explicáveis.

Ponto de partida

Reaplicar alternância de estado e DOM, presentes nos exemplos anteriores de cards e detalhes.

A pergunta seguinte depende da resposta

Quando a categoria for “outro”, o formulário pede uma explicação adicional. Quando a categoria mudar, o campo complementar deve deixar de ser obrigatório e não participar da coleta.

Esconder um elemento não significa desabilitar o controle. Ajuste o estado de forma explícita. Depois de voltar a “outro”, o controle deve funcionar novamente.

```
function atualizarOutro() {
  const ativo = categoria.value === "outro";
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = "";
}
categoria.addEventListener("change", atualizarOutro);
atualizarOutro();
```

Funções com uma responsabilidade

Separe atualizarOutro, conferirContato e gerarResumo. Dê nomes que indiquem o que cada função faz. Isso torna mais simples localizar erros e explicar uma alteração durante a apresentação.

Ao oferecer uma ação para começar um novo chamado, limpe também o resumo e os erros personalizados. Evite apagar o trabalho em andamento por meio de um botão de reset pouco claro.

Integração se verifica com mudanças de estado

Teste “outro”, preencha detalhes, volte a uma categoria comum e gere o resumo. Depois reabra “outro” e confirme se a regra continua adequada.

Confira novamente o layout e o teclado. Um comportamento novo pode criar uma barreira de uso em algo que já funcionava.

Atividade da aula

1. Implementar a categoria “outro” e seu campo complementar.
2. Atualizar hidden, disabled e required juntos.
3. Organizar o script em funções pequenas.
4. Executar os casos de ida, volta e novo preenchimento.

Checklist de conferência

[ ] O campo aparece apenas quando necessário.

[ ] Um campo desativado não bloqueia o envio.

[ ] A coleta reflete o estado atual do formulário.

[ ] Consigo explicar cada função do meu script.

Uma descoberta útil

Um campo com hidden no bloco pode continuar existindo no DOM. Um controle disabled normalmente fica fora de FormData.

Para ir além

Criar uma função que gere cada linha do resumo com createElement e textContent.

Referências da aula

MDN: atributo disabled

MDN: uso de FormData

MDN: evento submit

ENCONTRO 08 / 11/12 / Testes e publicação

O projeto nas mãos de outra pessoa

Revisão por pares, correções e publicação no GitHub Pages

O que você deverá conseguir

• Executar casos de teste com entradas e resultados esperados.

• Corrigir problemas com base em evidências.

• Publicar e repetir os testes no endereço público.

Ponto de partida

A publicação já aparece na trilha anterior. O foco agora é verificar o comportamento no endereço publicado e documentar o projeto.

Teste é uma pergunta verificável

Um caso descreve entrada, ação e resultado esperado. Inclua formulário vazio, formato de contato inadequado, contatos diferentes, descrição no limite, categoria “outro” e envio válido.

O colega registra o que observou e o autor tenta reproduzir o problema. Se houver uma correção, execute novamente os testes afetados. Não use dados privados de pessoas reais.

A publicação tem uma pasta de origem

Organize index.html, CSS, JavaScript e demais arquivos com caminhos relativos. Verifique letras maiúsculas e minúsculas nos nomes. Escreva no README como abrir e testar o projeto.

No GitHub Pages, selecione a origem de publicação disponível na sua conta e confira o resultado do processamento. Depois abra o endereço público e repita os testes principais.

Revisão técnica e explicação individual

Dê prioridade a funções quebradas e barreiras de uso antes de efeitos visuais adicionais. A apresentação final deve mostrar uma entrada válida, um erro e sua recuperação.

Cada aluno prepara uma explicação sobre HTML, uma regra CSS e uma parte do JavaScript. Quem trabalhar com ajuda continua responsável por explicar o próprio código.

Atividade da aula

1. Trocar o computador com um colega e executar seis casos de teste.

2. Registrar dois problemas ou oportunidades de melhoria.

3. Corrigir o que for necessário e repetir os casos afetados.

4. Publicar, conferir os links e preparar a demonstração final.

Checklist de conferência

[ ] Meus testes têm resultado esperado e observado.

[ ] A navegação por teclado e o celular foram conferidos.

[ ] A página pública carrega os arquivos corretamente.

[ ] O README informa o caráter simulado do projeto.

Uma descoberta útil

Um caminho que funciona no computador pode falhar na hospedagem se o nome do arquivo tiver letras maiúsculas e minúsculas diferentes.

Para ir além

Escrever no README uma limitação conhecida e uma melhoria futura viável.

Referências da aula

GitHub: configuração da publicação

GitHub: funcionamento do Pages

W3C WAI: notificações e erros

ENCONTRO 09 / 18/12 / Síntese

O que conseguimos construir e explicar

Demonstração, reavaliação e continuidade

O que você deverá conseguir

• Demonstrar o formulário e justificar decisões técnicas.

• Realizar uma pequena alteração com autonomia.

• Comparar a aprendizagem atual com a evidência de outubro.

Ponto de partida

Reabrir as respostas de 16/10 somente depois da nova tentativa individual, para observar a evolução sem copiar o raciocínio anterior.

Demonstração com evidência

Apresente o objetivo do formulário, preencha um caso válido e mostre o resumo. Em seguida, provoque um erro e demonstre a recuperação. Explique que o envio é simulado.

Mostre uma decisão de HTML e acessibilidade, uma de layout e uma de JavaScript. A fala deve acompanhar o código e o comportamento observado.

Uma alteração pequena, feita por você

Exemplos: acrescentar uma opção de categoria, mudar uma restrição com justificativa, alterar um texto do resumo ou ajustar um ponto de quebra. O professor escolhe uma mudança adequada ao trabalho e pede que o aluno teste o resultado.

A alteração ajuda a verificar compreensão. Não é uma corrida de digitação, e o professor pode oferecer uma pista registrada quando necessário.

Comparação e continuidade

Responda a uma nova versão curta das questões de HTML, CSS e JavaScript. Compare o que você explica agora com a primeira tentativa e registre uma evidência concreta de avanço.

O próximo percurso pode incluir requisições com fetch, API e backend com validação no servidor. Esses temas dependem da consolidação da base e ficam como continuidade, sem prometer que o protótipo atual já faz isso.

Atividade da aula

1. Demonstrar envio válido, erro e recuperação.
2. Explicar decisões de HTML, CSS e JavaScript.
3. Fazer uma pequena alteração individual e testá-la.
4. Registrar o que aprendeu e uma pergunta para continuar estudando.

Checklist de conferência

[ ] Demonstrei o formulário em vez de apenas mostrar a tela.

[ ] Expliquei uma regra e seu efeito no código.

[ ] Testei a alteração individual.

[ ] Comparei minha aprendizagem com a evidência inicial.

Uma descoberta útil

Explicar uma alteração pequena costuma revelar se compreendemos a relação entre seletor, evento e resultado na página.

Para ir além

Desenhar a API de uma versão futura, descrevendo os dados enviados e a resposta esperada.

Referências da aula

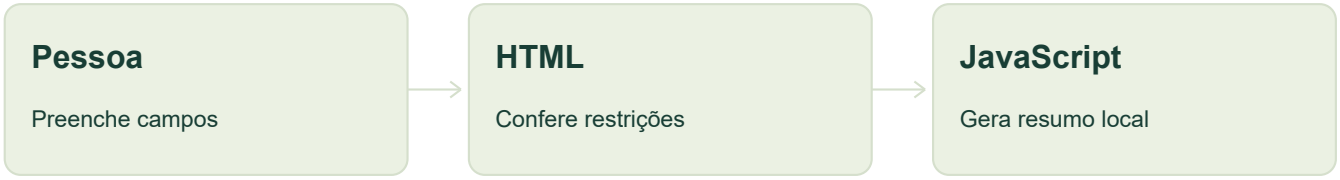
MDN: seu primeiro formulário

MDN: uso de FormData

MDN: validação de formulários

GitHub: funcionamento do Pages

Mapa do percurso dos dados



No protótipo: os dados ficam no navegador.

Em um sistema real, uma requisição iria a um servidor para nova validação e tratamento.

Servidor, banco de dados e autenticação serão temas de continuidade.

Seis testes do projeto

Caso	Resultado esperado
Campos obrigatórios vazios	O formulário orienta o preenchimento e não gera confirmação válida.
E-mail sem formato adequado	O controle indica o problema de formato.
Contatos diferentes	A confirmação pede o mesmo contato e permite correção.
Descrição curta e no limite	A regra de tamanho se comporta conforme a instrução.
Categoria outro e retorno	Campo complementar aparece, torna-se obrigatório e sai da coleta quando desativado.
Entrada válida	Resumo local mostra os valores e informa envio simulado.

Registro do teste

Para cada caso, anote: entrada utilizada, ação, resultado esperado, resultado observado e correção. Depois de corrigir, repita o teste.

Glossário de consulta

Termo	Função no projeto
DOM	Representação da página que o JavaScript consulta e altera.
id	Identifica um elemento no documento.
name	Define a chave de um dado na coleta do formulário.
label	Rótulo que identifica um controle.
submit	Evento associado à tentativa de envio do formulário.
preventDefault	Cancela a ação padrão de um evento cancelável.
FormData	Organiza entradas elegíveis de um formulário por nome.
Validação	Conferência de uma regra sobre os valores.
Backend	Parte do sistema que recebe e trata requisições em servidor.
HTTPS	Proteção do tráfego na conexão.
Responsividade	Adaptação do layout a diferentes tamanhos de tela.

Referências para continuar

Consulta das fontes em 15/10/2026. O portal anterior serve de evidência dos temas apresentados, não de comprovação do domínio individual. O texto e as práticas deste caderno são autorais.

Base anterior: portal do Prof. Ábner Lucas

MDN: seu primeiro formulário

MDN: tipos de input

W3C WAI: rótulos dos controles

W3C WAI: agrupamento de controles

MDN: validação de formulários

MDN: estilização de formulários

MDN: fundamentos de CSS Grid

MDN: evento submit

MDN: uso de FormData

MDN: setCustomValidity

W3C WAI: notificações e erros

MDN: envio de dados

MDN: elemento button

MDN: atributo disabled

GitHub: funcionamento do Pages

GitHub: configuração da publicação

MDN: textContent

Meu registro final

O que consigo fazer com autonomia? Qual evidência mostra isso? O que quero estudar na próxima etapa?

_____

_____